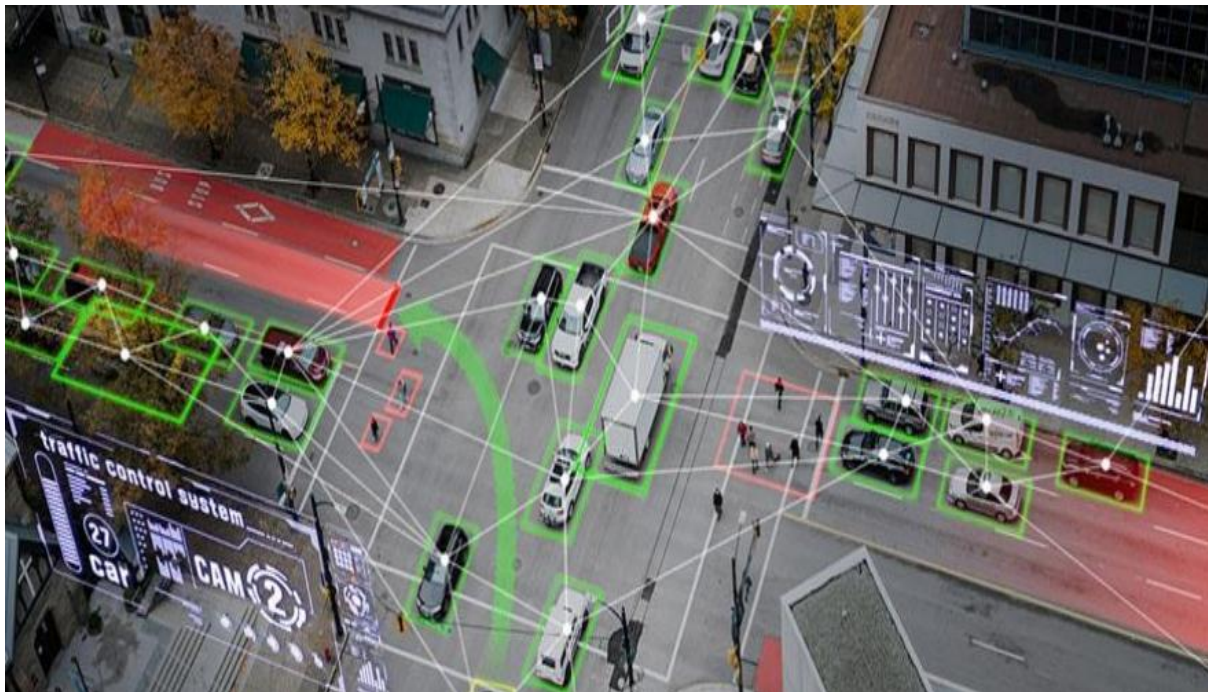


PROJECT TITLE: SMART CITY MANAGEMENT SYSTEM

DATA STRUCTURES & ALGORITHMS (CS-2001)

1. Executive Summary

This project simulates the complex infrastructure of Islamabad (Transport, Medical, Education, and Housing). Unlike standard implementations, we utilized **Custom C++ Templates** for our core data structures. This report details the architectural decisions behind these structures, specifically focusing on the trade-offs considered during implementation to achieve optimal time and space complexity.



2. Justification & Comparative Analysis

2.1 THE CITY MAP: GRAPH WITH ADJACENCY LIST

- **Usage:** Implemented in `CityGraph.h` to model intersections (Nodes) and roads (Weighted Edges).
- **Reason for Selection:** The Adjacency List (in the form of a Vector of LinkedLists) was used. The nature of a city road network is that it is a sparse graph; an intersection can hardly be connected to more than 3 or 4 other intersections. This graph is only efficient in storing valid connections $O(V+E)$.

2.2 HIERARCHY MANAGEMENT: N-ARY TREE

- **Usage:** Implemented in `NaryTree.h` for the Islamabad Administrative Structure (Sector -> Street -> House).
- **Reason for Selection:** Hierarchies in the real world are hardly binary. There are no two streets of a city sector such as F-10. The N-ary Tree enables a parent node to contain a dynamic Vector of children pointers, which is a good model of one-to-many relation of city zoning.

2.3 INSTANT RETRIEVAL: HASH TABLE WITH SEPARATE CHAINING

- **Usage:** Implemented in `HashTable.h` to find Medicines by Name and Citizens by CNIC.
- **Reason for Selection:** We must have $O(1)$ (average time) average access. List searching of 100,000 citizens is computationally costly. We have used Separate Chaining (we used `LinkedLists.h`) to manage the occurrence of collisions.

2.4 DECISION MAKING: PRIORITY QUEUE (BINARY MIN-HEAP)

- **Usage:** Implemented in `PriorityQueue.h` for the **Emergency Bed System** (Medical) and **School Ranking** (Education).
- **Reason for Selection:** We do not require a sorted list of hospitals in case of an emergency; all we require is the single hospital that has the highest availability at that particular

instance. A Binary Heap offers $O(1)$ access to the top element and efficacious $O(\log N)$ insertion/deletion.

2.5 ROUTE MANAGEMENT: SINGLY LINKED LIST

- **Usage:** Implemented in `LinkedLists.h` for Bus routes (Stop A -> Stop B -> Stop C).
- **Reason for Selection:** Bus services are line sequences. A Linked List enables the addition or deletion of a bus stop in the middle of a route in constant $O(1)$ time (assuming we have a pointer to the last node) without reallocating any memory.

2.6 PASSENGER FLOW: CIRCULAR QUEUE

- **Usage:** Implemented in `CircularQueue.h` for passengers waiting at a bus stop.
- **Reason for Selection:** A Queue is a structure that upholds the **FIFO (First-In-First-Out)** principle. To control the fixed buffer effectively and avoid the memory waste associated with standard linear queues, we employed a Circular implementation.

2.7 NAVIGATION HISTORY: STACK

- **Usage:** Implemented in `Stack.h` for tracking the user's navigation history (Backtracking).
- **Reason for Selection:** The nature of navigation is always **LIFO (Last-In-First-Out)**. The final place visited should be the initial one revisited when the user clicks "Back," making the Stack the only logical choice for this feature.

3. Complexity Analysis Summary

Component	Data Structure	Operation	Time Complexity
Route Finding	Graph (Adj List) + Heap	Dijkstra's Algorithm	$O((V + E) \log V)$
Emergency Bed	Priority Queue (Heap)	Get Nearest/Best	$O(1)$
Emergency Bed	Priority Queue (Heap)	Update Availability	$O(\log N)$
Medicine Search	Hash Table	Search (Avg Case)	$O(1)$
Medicine Search	Hash Table	Search (Worst Case)	$O(N)$
City Search	N-ary Tree	Traversal	$O(N)$
Bus Route	Linked List	Insert Stop	$O(1)$
Passenger Wait	Circular Queue	Enqueue/Dequeue	$O(1)$

4. Conclusion & Technical Achievements

This project successfully replicates a Smart City ecosystem by strictly adhering to modular design principles. By manually implementing core structures like HashTable, CityGraph, and PriorityQueue rather than relying on the STL, we gained fine-grained control over memory and performance. The use of C++ Templates further enhanced the project, allowing us to strictly follow the DRY (Don't Repeat Yourself) principle, resulting in a codebase that is both scalable and maintainable.